Лекция: Принципы проектирования программных систем (SOLID и GRASP)

Цель лекции: Перейти от понимания синтаксиса классов к архитектурному мышлению. Научиться писать код, который легко расширять, тестировать и поддерживать.

---

1. Введение: Зачем нам принципы?

В программировании есть понятие **STUPID** (Singleton, Tight Coupling, Untestability, Premature Optimization, Indescriptive Naming, Duplication). Чтобы не писать "глупый" код, мы используем проверенные временем практики.

Зачем это нужно?

- **Снижение стоимости изменений:** В коммерческой разработке 80% времени тратится на поддержку, а не на написание нового кода.
- **Управление сложностью:** Проекты растут, и без структуры они превращаются в «большой ком грязи».
- **Борьба с "хрупкостью":** Изменение в одном месте не должно ломать логику в другом.

---

2. Принципы SOLID

Это фундамент объектно-ориентированного дизайна, популяризированный Робертом Мартином (Uncle Bob).

S — Single Responsibility Principle (Принцип единственной ответственности)

У класса должна быть только одна причина для изменения.

✕ Нарушение (C#):
Класс `Report` делает всё: считает данные и сохраняет их в файл.

codeC#
```
public class Report {
    public string Content { get; set; }
    public void Generate() { /* логика генерации */ }
    // ОШИБКА: Класс не должен знать о файловой системе
    public void SaveToFile(string path) { File.WriteAllText(path, Content); }
}
```

✓ Соблюдение:
Разделяем логику и сохранение.

codeC#

```
public class Report { public void Generate() { ... } }
public class ReportSaver { public void Save(string path, Report report) { ... }
}
```

O — Open/Closed Principle (Принцип открытости/закрытости)

Программные сущности должны быть открыты для расширения, но закрыты для модификации.

❌ Нарушение:
Если нам нужно добавить новый тип скидки, придется менять основной класс `Calculator` (добавлять `if` или `switch`).

codeC#
```
public double CalculateDiscount(string type) {
    if (type == "Silver") return 0.1;
    if (type == "Gold") return 0.2; // Придется постоянно менять этот код
    return 0;
}
```

✅ Соблюдение:
Используем полиморфизм (интерфейсы или абстрактные классы).

codeC#
```
public interface IDiscount { double GetValue(); }
public class GoldDiscount : IDiscount { public double GetValue() => 0.2; }
// Теперь мы добавляем новые классы, не меняя существующий калькулятор.
```

L — Liskov Substitution Principle (Принцип подстановки Лисков)

Объекты в программе должны быть заменяемы на экземпляры их подтипов без изменения правильности работы программы.

Классический пример: Квадрат и Прямоугольник. Если `Square` наследует `Rectangle` и меняет логику установки сторон, он может сломать код, ожидающий поведения прямоугольника.

Ключевая мысль: Наследник не должен ограничивать поведение родителя (например, выбрасывать `NotImplementedException` там, где у родителя был рабочий метод).

I — Interface Segregation Principle (Принцип разделения интерфейса)

Клиенты не должны зависеть от методов, которые они не используют.

❌ Нарушение:
Огромный интерфейс `ISmartDevice`, заставляющий простую лампочку реализовывать метод `Print()`.

✅ Соблюдение:
Разделение на `IPowerable`, `IPrintable`, `IScannable`.
*В Python этот принцип менее строгий из-за **Duck Typing**, но в C# это критично для чистоты API.*

D — Dependency Inversion Principle (Принцип инверсии зависимостей)

Модули верхнего уровня не должны зависеть от модулей нижнего уровня. Оба должны зависеть от абстракций.

❌ **Нарушение:**
Класс `OrderService` внутри создает `new SqlDatabase()`. Теперь мы привязаны к SQL навечно.

✅ **Соблюдение:**

codeC#
```
public class OrderService {
    private readonly IDatabase _db;
    public OrderService(IDatabase db) { _db = db; } // Dependency Injection
}
```

GRASP, DRY, KISS и другие

Введение:
Представьте, что вы строите дом. Если вы просто свалите кирпичи в кучу, они не станут стеной. Если вы склеите их суперклеем намертво, вы не сможете потом провести проводку. В программировании всё так же: код должен быть структурированным, чтобы его можно было менять, не разрушая всё здание.

---

1. Принципы GRASP: Распределяем обязанности

Если SOLID говорит нам, какими должны быть классы, то **GRASP** отвечает на вопрос: «Где написать этот метод?».

1.1. Information Expert (Информационный эксперт)

Принцип: Метод должен быть в том классе, у которого больше всего данных для его выполнения.

❌ **Плохо:** Мы считаем сумму заказа в классе `CheckOutService`, вытягивая данные из заказа.

codeC#
```
// Сервис "ворует" данные у объектов
public class CheckOutService {
    public double CalculateTotal(Order order) {
        double total = 0;
        foreach (var item in order.Items) {
            total += item.Price * item.Quantity; // Логика снаружи данных
        }
        return total;
    }
}
```

✅ **Хорошо:** `Order` сам знает свои товары, пусть он и считает сумму.

codeC#

```
public class Order {
    public List<OrderItem> Items { get; set; }
    // Order - эксперт в своих данных
    public double GetTotal() => Items.Sum(x => x.Price * x.Quantity);
}
```

1.2. Creator (Создатель)

Принцип: Кто должен создавать объект? Тот, кто его использует или содержит.

❌ **Плохо:** Мы создаем элементы заказа где-то в UI и передаем их.
✅ **Хорошо:** Заказ (`Order`) содержит список элементов (`OrderItem`), значит, он их и должен создавать.

codeC#

```
public class Order {
    private List<OrderItem> _items = new();

    public void AddItem(string name, double price, int qty) {
        // Заказ сам создает свои части
        _items.Add(new OrderItem(name, price, qty));
    }
}
```

1.3. Controller (Контроллер)

Принцип: Должен быть объект-посредник, который принимает события от UI и передает их логике.

- **Отсылка к Python:** Это как `views.py` в Django. Он не считает налоги, он просто вызывает нужный метод модели.

codeC#

```
// UI только ловит клик и зовет контроллер
public class OrderController {
    public void OnClickBuy(int userId, int productId) {
        _orderService.CreateOrder(userId, productId); // Делегируем работу
    }
}
```

1.4. Low Coupling (Слабая связанность)

Принцип: Классы должны как можно меньше знать друг о друге.

❌ **Плохо:** Класс жестко привязан к конкретной базе данных.

codeC#

```
public class UserService {
    private MySqlDatabase _db = new MySqlDatabase(); // Жесткая связь
}
```

✅ **Хорошо:** Используем интерфейс. Теперь `UserService` не важно, какая БД используется.

codeC#

```
public class UserService {
    private IDatabase _db;
    public UserService(IDatabase db) => _db = db; // Слабая связь через
интерфейс
}
```

1.5. High Cohesion (Высокая сплоченность)

Принцип: Один класс — одна узкая задача. (Похоже на SRP из SOLID).

❌ **Плохо:** Класс-комбайн.

codeC#

```
public class User {
    public void SaveToDb() { ... }
    public void SendEmail() { ... }
    public void CalculateSalary() { ... }
}
```

Если в классе 500 строк и методы делают совершенно разные вещи — это низкая сплоченность.

1.6. Polymorphism (Полиморфизм)

Принцип: Если поведение зависит от типа — используй интерфейсы, а не `if/else`.

❌ **Плохо:**

codeC#

```
if (bird.Type == "Duck") Quack();
else if (bird.Type == "Owl") Hoot();
```

✅ **Хорошо:**

codeC#

```
public interface IBird { void MakeSound(); }
```

```
public class Duck : IBird { public void MakeSound() =>
Console.WriteLine("Quack"); }
```

1.7. Pure Fabrication (Чистая выдумка)

Принцип: Если логика не вписывается ни в один реальный объект — придумай искусственный класс.

- **Пример:** `Logger`, `Repository`. В жизни нет объекта «Репозиторий», но в коде он нужен для чистоты.

1.8. Indirection (Посредник)

Принцип: Чтобы два класса не зависели друг от друга напрямую, вставь между ними третий.

- *Пример:* Контроллер в MVC является посредником между Данными (Model) и Экраном (View).

1.9. Protected Variations (Устойчивость к изменениям)

Принцип: Оборачивай нестабильные части системы в интерфейсы.

- **Пример:** Если вы используете сторонний API для отправки СМС, сделайте свой интерфейс `ISmsSender`. Если сервис СМС сменится, вы поменяете только один класс, а не всю программу.

---

2. Практические принципы: DRY, KISS, YAGNI, LoD

2.1. DRY — Don't Repeat Yourself (Не повторяйся)

Суть: Не копируй код. Копипаст — это технический долг.

✕ Плохо:

codeC#

```
// В одном месте
var priceWithTax = price * 1.2;
// В другом месте через 100 строк
var finalPrice = total * 1.2;
```

✅ **Хорошо:** Вынеси коэффициент налога в константу или метод. Если налог станет 13%, ты поменяешь его в одном месте, а не в двадцати.

2.2. KISS — Keep It Simple, Stupid (Будь проще)

Суть: Самое элегантное решение — самое простое.

✕ **Плохо:** Использовать сложный LINQ-запрос там, где обычный `if` понятнее.

codeC#

```
// Чересчур "умный" код, который сложно читать
var result = list.Where(x => x.IsActive).SelectMany(x =>
x.Roles).Distinct().OrderBy(x => x).ToList();
```

Совет студентам: Пишите код так, будто сопровождать его будет маньяк-психопат, который знает, где вы живете.

2.3. YAGNI — You Ain't Gonna Need It (Тебе это не понадобится)

Суть: Не пиши код на будущее.

Пример: Заказчик просит регистрацию по Email. Программист думает: «А вдруг завтра захотят через Telegram, Facebook и по отпечатку пальца?» и пишет сложнейшую систему аутентификации на неделю.
Итог: Заказчик передумал, а время потрачено. Делайте только то, что нужно сейчас.

2.4. LoD — Law of Demeter (Закон Деметры)

Суть: «Не разговаривай с незнакомцами». Объект должен работать только с теми, кого знает лично.

✕ **Плохо:** «Железнодорожная авария» (Train Wreck).

codeC#

```
// Мы лезем через три колена вглубь чужих объектов
string city = student.Group.Department.Faculty.Dean.Address.City;
```

✓ **Хорошо:**

codeC#

```
string city = student.GetDeanCity(); // Студент сам спросит у кого надо
```

Почему это важно? Если у Декана изменится структура адреса, в «плохом» примере ваш код сломается. В «хорошем» — нет.

Резюме

1. **SOLID** — это про то, как строить классы.

2. **GRASP** — это про то, как распределять ответственность.
3. **KISS/DRY/YAGNI** — это про здравый смысл.

Python-отсылка: В Python есть документ **Zen of Python** (`import this`). Там есть фраза: *"Simple is better than complex"* (Простое лучше сложного). Это и есть KISS в действии. В C# мы стремимся к тому же, используя инструменты строгой типизации.